PARALLEL AND DISTRIBUTED SYSTEM · INITIAL FRAMEWORK REVIEW

Biliran Resort & Tour Booking System

Online and face-to-face booking of island packages — one inventory, no double booking, works offline

Member A · Member B · Member C · Member D

Instructor: Engr. Eunel O. Peñaranda · 30 October 2026

Problem and objectives

Biliran's resorts and tours sell to online travellers and walk-in guests at the same time

Problem

- Online and walk-in sales draw from the **same rooms, seats and boats** — the last unit can be sold twice
- Island links to the mainland are **unreliable**; a central-only system stops the front desk
- Each resort manages its own services, but registration, accounts and reports need one place
- Searching islands one after another gets slower with every island

Objectives

- Book packages **online (app)** and **face-to-face (desk)** from one inventory
- **Zero** oversold units and double check-ins under concurrency
- Islands keep operating **offline** and sync automatically
- **Measure** parallel speedup: search fan-out and reports

Scope and limitations

Prototype demonstrated on 4 laptops and 4 phones over one hotspot

In scope

- 3 island nodes (Biliran mainland, Maripipi, Higatangan) + 1 central node
- Packages per island (room + tour + boat + meal)
- Online hold → pay → confirm; walk-in booking at the desk
- Arrival confirmation: booking info + physical ID check
- Receptionist marks services unavailable
- Central admin, node admins, receptionists

Limitations

- Payment is **simulated**
- Packages cannot span islands
- Offline island → its packages not sold online
- No customer accounts (reference + e-mail)
- Only last 4 ID digits stored (RA 10173)
- Fictional businesses, sample prices

Parallel and distributed components

Where the course concepts live in the system

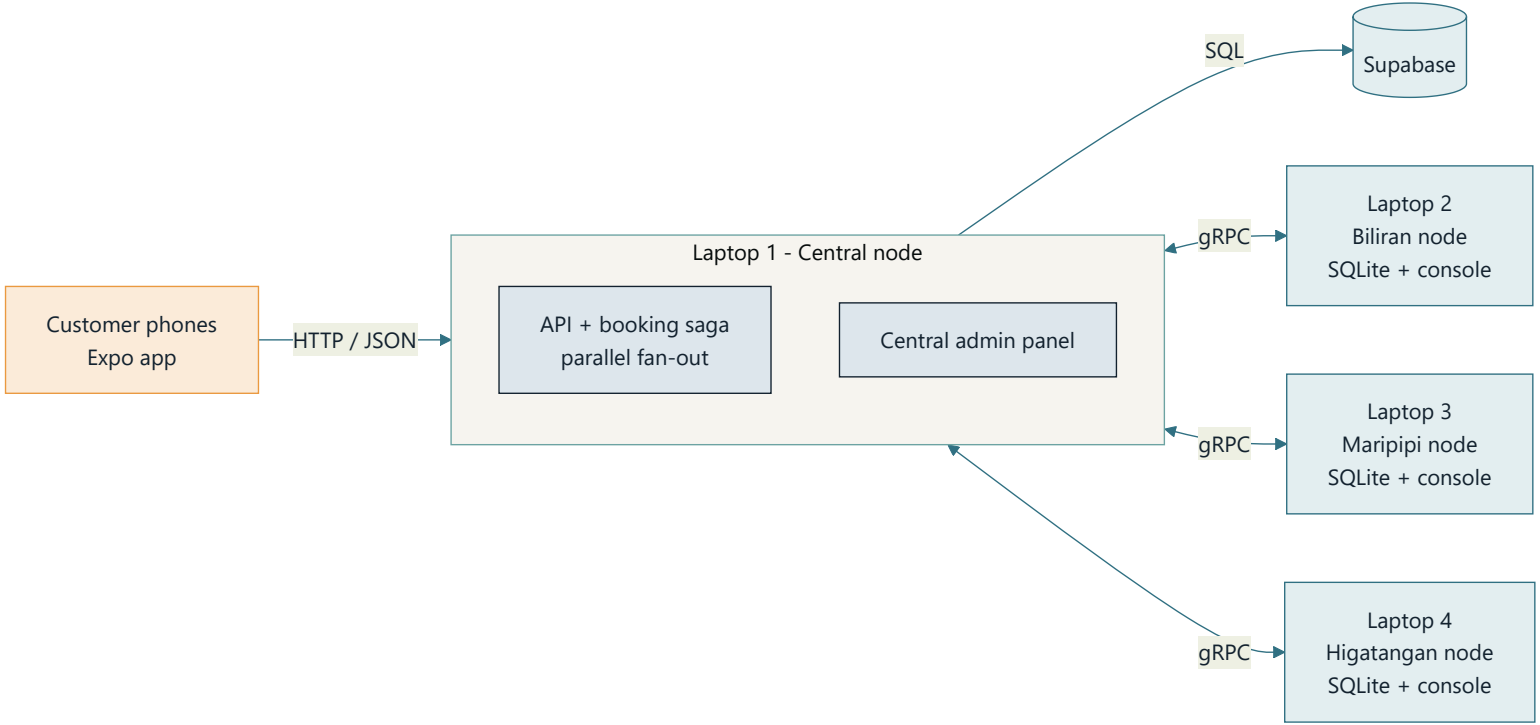
Distributed

- Data **partitioned by island**: each node owns its services, inventory and bookings
- Nodes communicate by **remote procedure calls (gRPC)** with deadlines
- **Outbox sync** with retries and de-duplication
- **Partition tolerance**: islands work offline, sync on reconnect
- One writer per data item — no conflicting edits

Parallel / concurrent

- **Parallel fan-out**: central queries all islands at once
- **Concurrent bookings** made safe with atomic guarded updates and transactions
- **Worker threads** compute occupancy and sales reports (1/2/4/8)
- **Load generators** fire N simultaneous requests to test and measure

System architecture



Conceptual framework (IPO)

INPUT

- Customer: date, guests, package, guest details, payment
- Receptionist: walk-ins, arrivals, unavailable services
- Node admin: services, packages, prices, receptionists
- Central admin: islands, node admins, report requests
- Network state: island online / offline

→

PROCESS

- Parallel search fan-out over gRPC
- Atomic hold / confirm on the owning island
- Saga: hold → pay → confirm, release on failure
- Idempotent requests; single writer per item
- Outbox sync and directory pull
- Offline operation of island nodes
- Parallel reports with worker threads

→

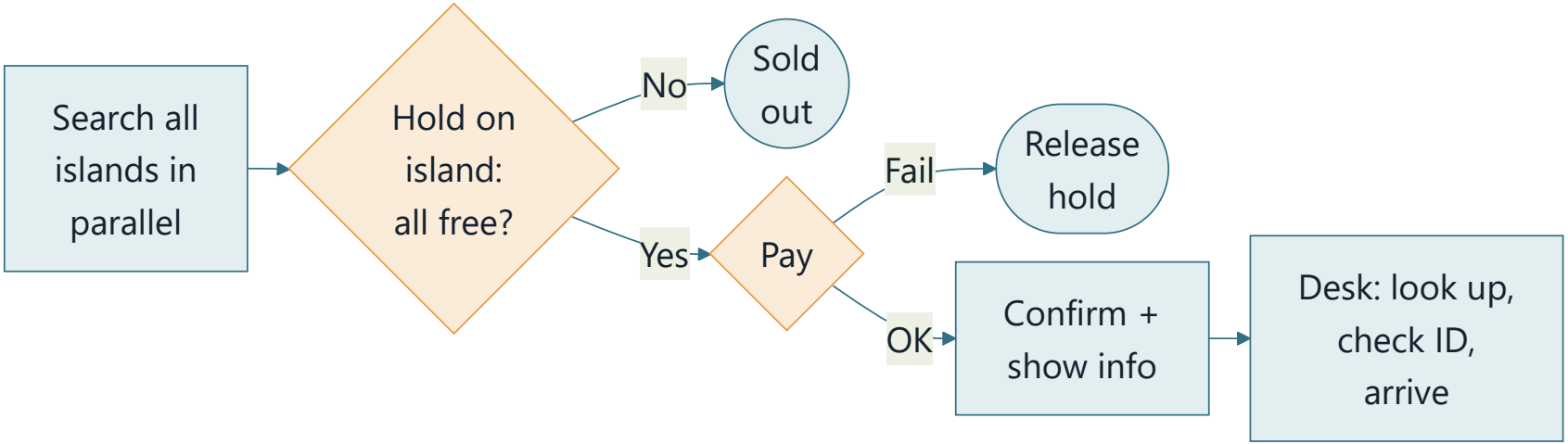
OUTPUT

- Confirmed bookings + confirmation info
- Inventory never oversold
- Guests checked in exactly once
- Province-wide reports
- Measured speedup, throughput, latency

Feedback: race, load, partition and speedup tests refine the design.

Main workflow: online booking

Walk-ins use the same island inventory directly from the node console



Consistency and offline behaviour

Race	Solution
Online vs walk-in, last room	Both go to the owning island; one atomic guarded update decides
Block a room while it is booked	Same rule: booked + blocked ≤ capacity
Retry after timeout	Idempotency key returns the same booking
Two desks confirm one arrival	CONFIRMED → CHECKED_IN succeeds once
Sync event delivered twice	De-duplicated by event id

Island offline

- Walk-ins, arrivals, logins keep working
- Changes queue in the outbox
- Its packages show **unavailable online** — never sell what the island cannot confirm (consistency over availability)
- On reconnect: outbox drains, directory pulled

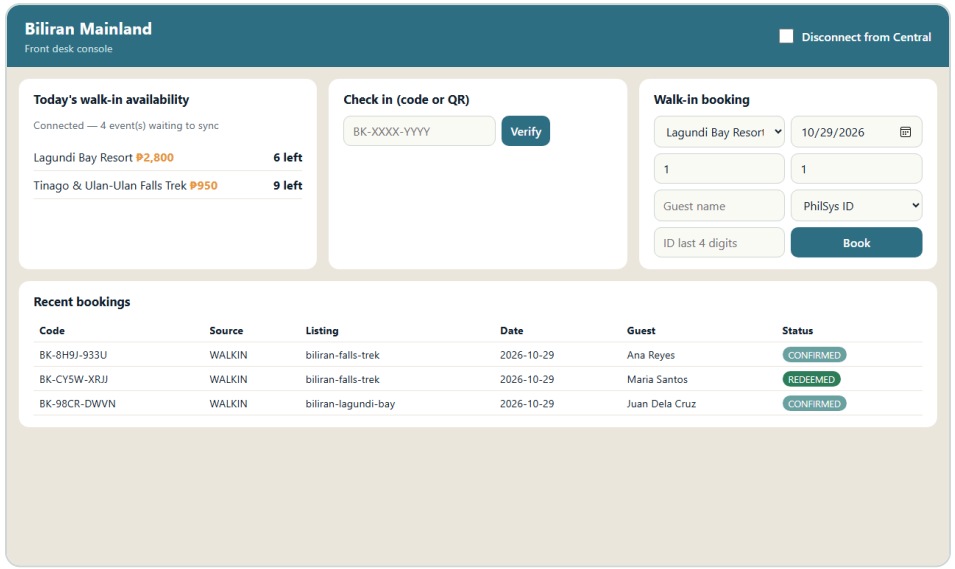
Modules and technology stack

Module	Owner
M1 Customer mobile app	Member C
M2–M5 Central API, orchestrator, admin panel, sync	Member A
M6–M8 Island booking engine, catalog & accounts, sync agent	Member B
M9 Node console (receptionist, node admin)	Member D
M10 Report workers · M11 Test & benchmark tools	Member D

Layer	Technology
App	Expo (React Native, TypeScript)
Servers	Node.js + TypeScript, Express
Node-to-node	gRPC (Protocol Buffers)
Island storage	SQLite (`node:sqlite`)
Central storage	Supabase PostgreSQL
Parallel reports	`worker_threads`

Status and early evidence

Baseline prototype (v1) running; reworking to the adviser-approved v2 design



2.93×
parallel vs sequential search across 3 islands (150 ms simulated link, 10 runs)

0 oversold
30 concurrent bookings for 6 rooms → exactly 6 booked

1 of 2
parallel double check-in → exactly one succeeds

Preliminary, measured on one laptop. Final measurements on the 4-laptop hotspot with 100 / 500 / 1,000 requests.

Next steps and items for approval

Plan to 20 November

- **Oct 30 – Nov 3:** v2 baseline, hotspot test with all 8 devices
- **Nov 2 – 10:** core features, gRPC v2, offline sync
- **Nov 9 – 13:** integration, tests, performance evidence
- **Nov 13 – 18:** documentation, slides, mock defense, fixes
- **Nov 19 – 20:** final defense and live demo

For approval

- Framework and architecture as presented
- Simulated payment (no real gateway)
- Offline island = unavailable online
- Package price per package (up to N guests)
- Cancellation: full refund up to 48 h before